`celecx`: Learning Curve Extrapolation for Batch-Sequential Computer Experiments

CELECX project

Abstract

`celecx` is short for Computer Experiment LEarning Curve eXtrapolation. The package is built for batch-sequential computer experiments where every new simulator evaluation has a cost, and where the relevant question is often how much progress is still expected from more evaluations.

The package has three parts. First, it provides active-learning and optimization machinery for running a sequential design and recording its archive. Second, it turns a run into a learning-curve task by evaluating surrogate-model quality after each batch. Third, it represents learning-curve extrapolation as ordinary `mlr3` machinery: tasks, learners, predictions, measures, resampling, and benchmark results. Extrapolation methods can therefore be compared in the same style as regression learners, and the extrapolation problem remains connected to the active-learning policy that generated the data.

The examples below use small synthetic experiments. They are meant to exercise the package contracts and the benchmark setup. The same code structure applies to expensive simulator traces and pre-evaluated finite pools.

Background

We consider a computer experiment with input space X and an expensive output $y = f(x)$. In practice an experiment usually starts with an initial design, often a Latin hypercube sample. An emulator or surrogate model is then fit on the evaluated points, many candidate points are generated, the candidates are ranked by an acquisition criterion, and a batch of new points is proposed for evaluation. After the simulator has evaluated the batch, the archive is updated and the process can be repeated.

A separate performance-estimation workflow is also common. One subsamples already available data at increasing sample sizes, fits a model on each prefix or subsample, measures out-of-sample performance, and extrapolates the resulting curve. This gives a rough answer to the question "how much more data is needed?". For active learning, the future data are generated by the acquisition policy. The learning curve should therefore be tied to the sequential-design strategy, because uncertainty sampling, random sampling, and input-space coverage can produce different future archives.

The central question in this report is:

> Given the current archive and the active-learning strategy, how many more batches are likely to be needed to reach a target surrogate performance?

The package answers this by recording batch-level surrogate performance during active learning and by modelling this batch-level curve.

Problem Setting

The search domain is represented by a `paradox::ParamSet`. It can contain continuous, integer, logical, categorical, and mixed parameters, and it can carry dependencies and transformations. The same object is used throughout the `mlr3` and `bbotk` ecosystem to describe search spaces.

The expensive objective is a `bbotk::Objective`. For optimization, codomain targets are tagged `"minimize"` or `"maximize"`. For pure active learning the target is tagged `"learn"`, meaning that the objective is observed and learned, but no single best value is reported by the optimizer. The archive contains evaluated configurations, observed outputs, timestamps, and batch numbers.

A surrogate is trained on the archive. It may be used as an emulator for prediction, as a model behind an acquisition function, or both. At the end of every batch we can score the surrogate on a held-out regression task. This gives a learning curve

$$b \mapsto m_b,$$

where b is the batch number and m_b is a scalar performance value such as MAE, RMSE, R^2, best-so-far objective value, or regret. The examples below use held-out MAE of the surrogate.

`TaskLCE` stores this curve together with enough provenance to make replay and simulation possible. The ordinary feature for a learning-curve learner is the future `batch_nr`. The archive columns are carried as special roles, because advanced extrapolators may inspect the archive during training, while prediction at future batches should require only the future batch numbers.

Package Overview

`celecx` extends the `mlr3` ecosystem. The surrounding packages are used for their normal jobs:

- `paradox` describes domains, codomains, and parameter sets.
- `bbotk` provides objectives, terminators, optimizers, and archive conventions.
- `mlr3mbo` provides surrogate and acquisition-function conventions.
- `mlr3` provides tasks, learners, predictions, measures, resampling, and benchmarking.

The package layers are:

- user-facing active-learning entry points: `optimize_active()`, `optimizer_active_learning()`, and `optimizer_pool_al()`;
- objective wrappers for simulator functions, pre-evaluated datasets, and learner-based replay;
- active-learning optimizers and component objects;
- surrogate uncertainty wrappers;
- callbacks and replay helpers that collect batch-level surrogate performance;
- `TaskLCE`, `LearnerLCE`, `PredictionLCE`, and `MeasureLCE`;
- helpers for target-reaching forecasts such as `lce_batches_to_target()`.

The ordinary workflow is:

1. Define an objective or finite pool.
2. Run active learning with `optimize_active()`.
3. Track surrogate performance with `CallbackSurrogatePerformance`.
4. Build a `TaskLCE`.
5. Fit LCE learners and evaluate them with LCE measures.
6. Convert a trained LCE learner into a batches-to-target forecast.

Most components are R6 objects. They are reference-based, so changing a field changes the object seen by all variables pointing to it. The package clones objects where aliasing would be surprising, in particular when tasks store search spaces, codomains, measures, and pools. When constructing components directly, the same rule as in other `mlr3` packages applies: use `$clone(deep = TRUE)` when an independent copy is needed.

A First Active-Learning Run

We start with a small one-dimensional objective. The codomain target is tagged `"learn"`, because this is an active-learning example. The held-out task is dense and cheap; it is used only to measure surrogate performance during the run.

```
set.seed(1L)

objective_fun = function(x) {
  sin(pi * x) + 0.15 * cos(3 * pi * x)
}

objective = ObjectiveRFun$new(
  fun = function(xs) {
    list(y = objective_fun(xs$x))
  },
  domain = ps(x = p_dbl(lower = 0, upper = 2)),
  codomain = ps(y = p_dbl(tags = "learn"))
)

test_data = data.table(x = seq(0, 2, length.out = 60L))
test_data[, y := objective_fun(x)]
test_task = as_task_regr(test_data, target = "y")
```

We attach a callback that evaluates the active-learning surrogate after every batch. The default uncertainty optimizer stores its surrogate under the id `"uncertainty"`. Here we use a small bootstrap ensemble around a regression tree, because this gives standard errors for acquisition scoring without relying on a Gaussian-process backend.

```
performance_callback = clbk("celecx.surrogate_performance",
  surrogate_id = "uncertainty",
  task = test_task,
  measures = list(mae = msr("regr.mae"))
)

surrogate_learner = lrn("regr.rpart", minsplit = 2L, cp = 0)

result = optimize_active(
  objective = objective,
  n_evals = 10L,
  learner = surrogate_learner,
  n_bootstrap = 4L,
  batch_size = 2L,
  acq_evals = 12L,
  callbacks = list(performance_callback)
)
```

`optimize_active()` returns the search instance and the optimizer. The instance owns the archive. The archive is the evaluated design together with observed outputs and batch numbers.

```
archive_data = result$instance$archive$data
archive_data[, .(x, y, batch_nr)]
#>             x          y batch_nr
#>         <num>      <num>    <int>
#>  1: 0.5310173  1.0384839        1
#>  2: 0.7442478  0.8314264        1
#>  3: 1.3740457 -0.7836355        2
#>  4: 0.7682074  0.7519220        2
#>  5: 1.4474219 -0.9150633        3
#>  6: 1.6418926 -1.0482092        3
#>  7: 1.2845765 -0.6451582        4
```

```
#>  8: 1.4222424 -0.8699628        4
#>  9: 1.0520554 -0.2951158        5
#> 10: 1.0152836 -0.1964431        5
```

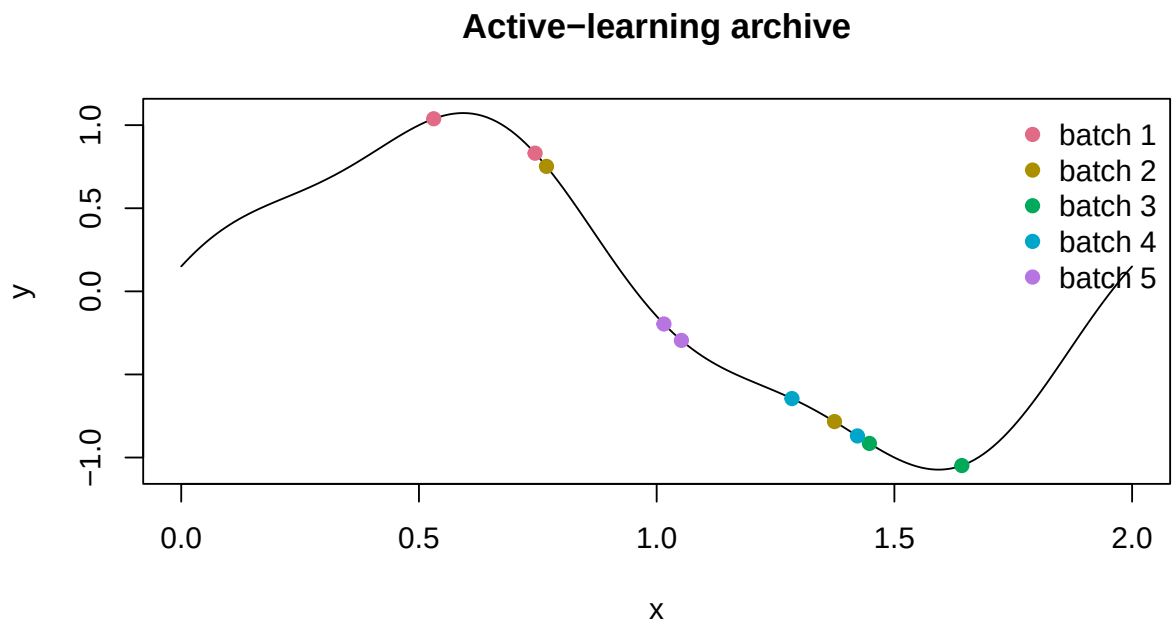
The callback holds one row per completed batch. The values below are the surrogate's held-out MAE after each batch.

```
performance_callback$data[, .(batch_nr, n_evals, surrogate_id, mae)]
#>    batch_nr n_evals surrogate_id       mae
#>       <int>   <int>       <char>     <num>
#> 1:        1       2  uncertainty 0.9630888
#> 2:        2       4  uncertainty 0.7386914
#> 3:        3       6  uncertainty 0.3340183
#> 4:        4       8  uncertainty 0.2871990
#> 5:        5      10  uncertainty 0.2637435
```

The plot shows the toy objective and the evaluated points. Points are colored by the batch in which they were evaluated.

```
curve_x = seq(0, 2, length.out = 200L)
curve_y = objective_fun(curve_x)
batch_cols = hcl.colors(max(archive_data$batch_nr), "Dark 3")

plot(curve_x, curve_y, type = "l",
  xlab = "x", ylab = "y", main = "Active-learning archive")
points(archive_data$x, archive_data$y,
  pch = 19, col = batch_cols[archive_data$batch_nr])
legend("topright", legend = paste("batch", seq_along(batch_cols)),
  col = batch_cols, pch = 19, bty = "n")
```



Conceptually the run follows the standard candidate-ranking workflow. An initial design is evaluated. A

surrogate is trained. Candidate points are generated and scored by predictive standard deviation. The best-scoring candidates are evaluated as a batch and appended to the archive.

User-Facing Active-Learning Interface

The public active-learning API has three nested entry points.

- `optimize_active()` runs a complete experiment.
- `optimizer_active_learning()` builds the default uncertainty-based optimizer used by `optimize_active()`.
- `optimizer_pool_al()` builds named pool-style active-learning methods for comparisons.

Use `optimize_active()` when the default uncertainty workflow is enough. Pass an optimizer from `optimizer_active_learning()` when the same strategy needs explicit construction, reuse, or inspection. Pass an optimizer from `optimizer_pool_al()` when comparing named finite-pool methods such as GSx, GSy, iGS, QBC, IDEAL, and random sampling. Constructing `OptimizerAL` directly mainly matters for new methods and is described later.

`optimize_active()`

`optimize_active()` receives a `bbotk::Objective`, constructs a `SearchInstance`, runs an optimizer until the terminator fires, and returns the objects needed for analysis. When `optimizer = NULL`, remaining arguments are passed to `optimizer_active_learning()`. This keeps the common uncertainty-based run compact:

```
result = optimize_active(
  objective = objective,
  n_evals = 40L,
  learner = lrn("regr.rpart"),
  se_method = "bootstrap",
  n_bootstrap = 30L,
  batch_size = 4L,
  acq_evals = 1000L
)
```

The result is a list with:

- `instance`, the `SearchInstance` containing objective, terminator, callbacks, and archive;
- `optimizer`, the configured active-learning optimizer;
- `metrics_tracker`, when the metrics-tracking interface is used.

User callbacks can be attached through the `callbacks` argument. For LCE work, `CallbackSurrogatePerformance` is the callback used most often, because it turns the run into a learning curve.

`optimizer_active_learning()`

`optimizer_active_learning()` builds an uncertainty-based optimizer. It is the direct implementation of "generate candidate points, rank them by surrogate uncertainty, select a batch".

The required input is a regression learner used as the surrogate. The standard-error method is selected with `se_method`:

- `"auto"` uses native standard errors when the learner supports them and otherwise uses bootstrap standard errors;
- `"bootstrap"` wraps the learner in `LearnerRegrBootstrapSE`;
- `"quantile"` wraps a quantile-prediction learner in `LearnerRegrQuantileSE`.

The main budget parameters are `batch_size` and `acq_evals`. `batch_size` is the number of proposed points per active-learning iteration. `acq_evals` is the number of candidates scored in each proposal round.

```
uncertainty_optimizer = optimizer_active_learning(
  learner = lrn("regr.rpart"),
  se_method = "bootstrap",
  n_bootstrap = 30L,
  batch_size = 4L,
  acq_evals = 1000L,
  multipoint_method = "greedy"
)

result = optimize_active(
  objective = objective,
  n_evals = 40L,
  optimizer = uncertainty_optimizer
)
```

The multipoint method determines how a batch is built:

- `"greedy"` scores candidates once and takes the top candidates;
- `"local_penalization"` builds the batch sequentially and penalizes points near already selected points;
- `"diversity"` builds the batch sequentially with an explicit diversity penalty;
- `"constant_liar"` uses pseudo-labels after each within-batch selection.

Candidate generation is controlled through `acq_optimizer`. Common optimizers are translated to `SpaceSampler` objects. For the typical LHS-candidate baseline, this means that Latin hypercube candidate generation is a sampler choice; the rest of the active-learning loop remains unchanged.

`optimizer_pool_al()`

`optimizer_pool_al()` builds named pool-based active-learning methods. The methods either score a finite pool exhaustively or score a uniformly subsampled candidate set each round through `pool_size`. This matches pre-evaluated datasets, replay studies, and finite libraries of feasible simulator inputs.

```
pool_optimizer = optimizer_pool_al(
  method = "igs",
  learner = lrn("regr.rpart"),
  n_init = 8L,
  batch_size = 4L,
  pool_size = 1000L
)

result = optimize_active(
  objective = pool_objective,
  n_evals = 80L,
  optimizer = pool_optimizer
)
```

The supported methods are:

- `"random"`, a random baseline;
- `"gsx"`, greedy sampling in input space;
- `"gsy"`, greedy sampling in predicted output space;
- `"igs"`, improved greedy sampling, combining input-space distance and predicted output differences;
- `"qbc"`, query-by-committee through bootstrap disagreement;
- `"ideal"`, inverse-distance active learning with exploration weight `delta`.

`"random"` and `"gsx"` do not require a learner. The methods `"gsy"`, `"igs"`, `"qbc"`, and `"ideal"` use a regression learner. Initialization can be controlled with `n_init` and `init_method`, where the available

initialization methods are `"gsx"`, `"random"`, and `"kmeans"`.

Objectives and Replay

The active-learning interface uses `bbotk` objectives. For an on-demand simulator, a regular objective such as `ObjectiveRFun` is enough. For development and benchmarks it is often better to replay a known table of evaluations. `ObjectiveDataset` wraps such a table as a finite-pool objective.

```
pool = data.table(
  x = seq(0, 2, length.out = 200L)
)
pool[, y := objective_fun(x)]

pool_objective = ObjectiveDataset$new(
  dataset = pool,
  domain = ps(x = p_dbl(lower = 0, upper = 2)),
  codomain = ps(y = p_dbl(tags = "learn"))
)

pool_result = optimize_active(
  objective = pool_objective,
  n_evals = 20L,
  optimizer = optimizer_pool_al(
    method = "gsx",
    n_init = 4L,
    batch_size = 2L
  )
)
```

`ObjectiveDataset` evaluates by table lookup and rejects configurations outside the table. Finite replay should use the same candidate library that would be available in the real analysis.

`ObjectiveLearner` is used when a fitted regression learner acts as a proxy objective. Forward simulation uses this pattern when the original simulator is replaced by an oracle model fit on the archive.

From an Optimization Trace to an LCE Task

An active-learning archive has one row per evaluated point. A learning curve has one value per batch. `TaskLCE` stores both in one task. The target column is the batch-level performance value; the ordinary feature is `batch_nr`; the original archive input and output columns are stored in the special roles `archive_x` and `archive_y`.

The callback from the first run can build such a task directly:

```
task_lce = performance_callback$task(measure = "mae", link = "log")
task_lce
#>
#> -- <TaskLCE> (10x2) ------------------------------------------------------------
#> * Target: mae
#> * Properties: -
#> * Features (1):
#>   * int (1): batch_nr
#> * Archive features: x
#> * Archive targets: y
```

The task has one row per archive evaluation, and all rows in the same batch share the same batch-level performance target.

```
task_lce$data()[, .(mae, batch_nr)][1:8]
#>          mae batch_nr
#>        <num>    <int>
#> 1: 0.9630888        1
#> 2: 0.9630888        1
#> 3: 0.7386914        2
#> 4: 0.7386914        2
#> 5: 0.3340183        3
#> 6: 0.3340183        3
#> 7: 0.2871990        4
#> 8: 0.2871990        4
```

The `link = "log"` argument says that the LCE model should represent uncertainty on the log scale. The log scale matches non-negative loss values. Point predictions remain on the natural scale, while standard errors are interpreted on the link scale.

Finished archives can also be replayed offline with `replay_surrogate_performance()`. The replay path refits a surrogate on each archive prefix, scores it on a held-out task, and returns the same `TaskLCE` shape.

```
replayed_task = replay_surrogate_performance(
  archive = result$instance$archive,
  learner = lrn("regr.rpart"),
  task = test_task,
  measures = list(mae = msr("regr.mae")),
  measure = "mae",
  link = "log"
)
```

The online callback and the offline replay helper cover different settings. The callback is convenient during a live run. Replay is the right choice when an archive already exists or when several surrogate choices should be scored on the same trace.

LCE Predictions

A `LearnerLCE` trains on `TaskLCE`. At prediction time it receives future `batch_nr` values. The simplest prediction is a point forecast of future performance. Many learners also report uncertainty.

```
lce_learner = lrn("lce.parametric_exponential")
lce_learner$predict_type = "se"
lce_learner$train(task_lce)

future_batches = as.integer((max(task_lce$batch_nrs) + 1L):
  (max(task_lce$batch_nrs) + 4L))

lce_prediction = lce_learner$predict_newdata(
  data.table(batch_nr = future_batches)
)

cbind(
  data.table(batch_nr = future_batches),
  as.data.table(lce_prediction)[, .(response, se, se_epistemic)]
)
#>    batch_nr  response        se se_epistemic
#>       <int>     <num>     <num>        <num>
#> 1:        6 0.2119118 0.3625815    0.2924430
#> 2:        7 0.1908450 0.4613416    0.4085262
```

```
#> 3:         8 0.1770398 0.5587837    0.5160395
#> 4:         9 0.1677592 0.6464557    0.6098873
```

The `PredictionLCE` object can carry:

- `response`, a point forecast on the natural scale;
- `se`, total predictive standard deviation on the link scale;
- `se_epistemic`, uncertainty of the mean curve on the link scale;
- `quantiles`, predictive quantiles;
- `samples`, joint predictive sample paths;
- `target_reached`, probabilities of reaching configured targets.

The distinction between `se` and `se_epistemic` matters. If a future batch is scored after it occurs, the realized performance includes residual variation, so total uncertainty is relevant. For planning the expected number of batches needed to reach a target, the uncertainty of the mean curve is often the relevant quantity.

LCE Learners

The LCE learner contract is the usual `mlr3` learner contract specialized to `task_type = "lce"`. The learner is trained on observed batch numbers and performance values. It predicts at future batch numbers. It declares which prediction types it supports, so the same learner can be used with point, interval, and distributional measures when available.

The package contains several learner families:

- `lce.featureless`, a baseline that predicts an average, last, or best observed value;
- `lce.rolling_slope`, a linear extrapolation from recent batches;
- `lce.parametric_exponential`, `lce.parametric_power_law`, `lce.parametric_log`, and `lce.parametric_logistic`, parametric curve models;
- `lce.isotonic` and `lce.spline_monotone`, monotone nonparametric fits;
- `lce.bootstrap`, a residual-bootstrap wrapper around another LCE learner;
- `lce.conformal`, a split-conformal wrapper around another LCE learner;
- `lce.simulate`, a forward-simulation learner that re-runs active learning on an oracle model fit to the archive.

The first two groups are fast baselines. The parametric learners extrapolate beyond the observed range by assuming a curve family. The monotone learners impose weak shape constraints. The wrappers add distributional information around a base learner. The simulation learner uses more of the stored active-learning provenance, because it forecasts by replaying the active-learning strategy forward.

LCE Measures and Resampling

LCE measures score predictions at the batch level. The simple measures are:

- `lce.mse`;
- `lce.rmse`;
- `lce.mae`.

Distributional measures use the uncertainty information in `PredictionLCE`:

- `lce.crps`, a closed-form CRPS for a Normal distribution on the link scale;
- `lce.reach_brier`, Brier score for target-reaching probabilities;
- `lce.coverage`, empirical coverage of central prediction intervals;
- `lce.interval_score`, interval width plus miscoverage penalty;
- `lce.pinball`, quantile loss for reported quantiles.

Random row resampling would leak future batches into the training set. `ResamplingLCE` therefore uses expanding windows over whole batches. Each split trains on a prefix, tests on the next `horizon` batches,

and advances by `step_size`. It can be used through `resample()` and `benchmark()` like any other `mlr3` resampling.

The following benchmark uses a synthetic `TaskLCE`. The task is constructed directly to keep the example short, but it has the same column roles as a task constructed from an active-learning trace.

```
make_lce_task = function(n_batches = 9L, per_batch = 2L) {
  set.seed(11L)
  batches = seq_len(n_batches)
  perf = 0.06 + 0.55 * exp(-0.35 * batches)
  perf = pmax(perf + rnorm(n_batches, sd = 0.01), 1e-3)
  n = n_batches * per_batch

  data = data.table(
    x1 = runif(n),
    x2 = runif(n),
    y = rnorm(n),
    batch_nr = as.integer(rep(batches, each = per_batch)),
    mae = rep(perf, each = per_batch)
  )

  TaskLCE$new(
    id = "synthetic_lce",
    backend = data,
    target = "mae",
    batch_nr = "batch_nr",
    archive_x = c("x1", "x2"),
    archive_y = "y",
    search_space = ps(x1 = p_dbl(0, 1), x2 = p_dbl(0, 1)),
    codomain = Codomain$new(ps(y = p_dbl(tags = "learn"))$domains),
    measure = msr("regr.mae"),
    link = "log"
  )
}

benchmark_task = make_lce_task()

benchmark_learners = lapply(
  c("lce.featureless", "lce.rolling_slope", "lce.parametric_exponential"),
  function(id) {
    learner = lrn(id)
    learner$predict_type = "se"
    learner
  }
)

benchmark_resampling = rsmp("lce.expanding_cv",
  horizon = 1L,
  min_train_batches = 4L,
  step_size = 1L
)

benchmark_design = benchmark_grid(
  tasks = benchmark_task,
  learners = benchmark_learners,
```

```
  resamplings = benchmark_resampling
)

benchmark_result = benchmark(benchmark_design)
benchmark_result$aggregate(list(msr("lce.mae"), msr("lce.crps")))[
  , .(learner_id, lce.mae, lce.crps)
]
#>                     learner_id    lce.mae   lce.crps
#>                         <char>      <num>      <num>
#> 1:            lce.featureless 0.02031567 0.14750423
#> 2:          lce.rolling_slope 0.01375073 0.09924017
#> 3: lce.parametric_exponential 0.01626350 0.09434277
```

The benchmark object can be inspected and post-processed with the usual `mlr3` tools. The important point is that LCE methods are ordinary learners and LCE scoring is ordinary benchmark scoring.

Batches-to-Target Forecasts

The forecast needed for the research question is often a target-crossing forecast. `lce_batches_to_target()` converts a trained LCE learner into a distribution over batch numbers at which a target is reached. The optimization direction is read from the measure stored in the training task. For a loss such as MAE, lower values are better.

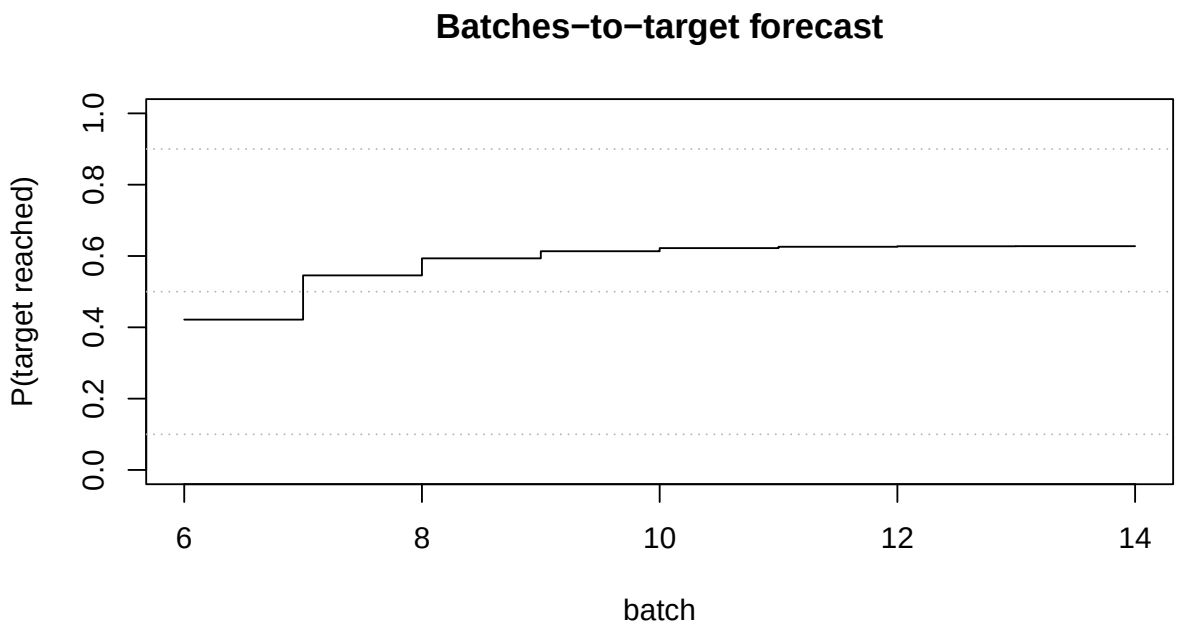
```
target_mae = 0.20
batch_grid = as.integer((max(task_lce$batch_nrs) + 1L):
  (max(task_lce$batch_nrs) + 9L))

batches_to_target = lce_batches_to_target(
  learner = lce_learner,
  batch_grid = batch_grid,
  target = target_mae,
  crossing = "expected"
)

batches_to_target$quantiles
#> q0.1 q0.5 q0.9
#>    6    7   NA
batches_to_target$p_never
#> [1] 0.3724535
batches_to_target$grid
#>    batch       cdf
#>    <int>     <num>
#> 1:     6 0.4215905
#> 2:     7 0.5456566
#> 3:     8 0.5934021
#> 4:     9 0.6134147
#> 5:    10 0.6221675
#> 6:    11 0.6259040
#> 7:    12 0.6272857
#> 8:    13 0.6275465
#> 9:    14 0.6275465
```

The grid component is a CDF over possible future crossing batches.

```r
plot(batches_to_target$grid$batch, batches_to_target$grid$cdf,
  type = "s", ylim = c(0, 1),
  xlab = "batch", ylab = "P(target reached)",
  main = "Batches-to-target forecast")
abline(h = c(0.1, 0.5, 0.9), lty = 3, col = "grey70")
```



There are two crossing semantics. `crossing = "expected"` asks when the de-noised mean curve reaches the target. It uses `se_epistemic`. `crossing = "observed"` asks when a realized noisy future curve reaches the target. It requires learners with joint sample paths, such as `lce.bootstrap` and `lce.simulate`.

Benchmark Study

The final benchmark has the same shape as the small example above. The difference is that the learning curves are produced by active-learning traces instead of by directly constructing a synthetic task.

The trace-generation part is:

1. Choose a simulator function, pre-evaluated pool, or learner proxy.
2. Choose an active-learning policy with `optimizer_active_learning()` or `optimizer_pool_al()`.
3. Run `optimize_active()` under a fixed budget.
4. Use `CallbackSurrogatePerformance` or `replay_surrogate_performance()` to construct one `TaskLCE` per trace.

The LCE benchmark part is:

1. Define LCE learners with comparable prediction types.
2. Use expanding-window splits over complete batches.
3. Score point forecasts, distributional forecasts, and target-reaching probabilities.
4. Aggregate by LCE learner, active-learning policy, simulator, and horizon.

The active-learning methods of interest are random sampling, uncertainty sampling, GSx, GSy, iGS, QBC, IDEAL, and an LHS-candidate uncertainty baseline matching the practical candidate-generation workflow.

The LCE methods of interest are featureless and rolling-slope baselines, parametric exponential and power-law curves, monotone methods, residual bootstrap, conformal bands, and forward simulation.

```
trace_methods = list(
  random = optimizer_pool_al("random", n_init = 8L, batch_size = 4L),
  gsx = optimizer_pool_al("gsx", n_init = 8L, batch_size = 4L),
  igs = optimizer_pool_al("igs",
    learner = lrn("regr.rpart"),
    n_init = 8L,
    batch_size = 4L,
    pool_size = 1000L
  ),
  uncertainty = optimizer_active_learning(
    learner = lrn("regr.rpart"),
    se_method = "bootstrap",
    n_bootstrap = 30L,
    batch_size = 4L,
    acq_evals = 1000L
  )
)

lce_methods = list(
  lrn("lce.featureless"),
  lrn("lce.rolling_slope", window = 4L),
  lrn("lce.parametric_exponential"),
  lrn("lce.parametric_power_law"),
  lrn("lce.isotonic", direction = "decreasing"),
  lrn("lce.bootstrap", learner = lrn("lce.parametric_exponential"))
)

for (learner in lce_methods) {
  learner$predict_type = if ("se" %in% learner$predict_types) "se" else "response"
}

resampling = rsmp("lce.expanding_cv",
  min_train_batches = 6L,
  horizon = 2L,
  step_size = 1L
)

design = benchmark_grid(lce_tasks, lce_methods, resampling)
bmr = benchmark(design)
bmr$aggregate(list(
  msr("lce.mae"),
  msr("lce.rmse"),
  msr("lce.crps"),
  msr("lce.coverage"),
  msr("lce.interval_score")
))
```

The output tables answer different questions. `lce.mae` and `lce.rmse` measure point-forecast accuracy. `lce.crps`, `lce.coverage`, and `lce.interval_score` evaluate distributional information. `lce.reach_brier` evaluates a target-reaching decision. Runtime per forecast should also be reported, because simulation and bootstrap methods can be more expensive than parametric curves.

Relation to the Research Project

The project studies whether batch-sequential experimental design can be improved by forecasting progress. The package separates this into two connected problems.

The first problem is how to generate the active-learning traces. The high-level optimizer factories express the scientific choices in a reusable way: surrogate model, uncertainty estimate, batch size, candidate-generation budget, finite-pool method, and initialization method. Active-learning policies can then be compared under the same objective and budget.

The second problem is how to forecast progress from a trace. `TaskLCE` turns a trace into a supervised learning problem over batch numbers. `LearnerLCE` objects then become comparable forecasting methods. This changes learning-curve extrapolation from a bespoke post-processing script into a benchmarkable modelling problem.

The examples in this report are single-target. The underlying objective and codomain machinery is compatible with richer output structures, but joint multi-output progress criteria need to be chosen carefully. For the present report the main target is surrogate performance for one learned output.

In-Depth: Active-Learning Components

The high-level functions are wrappers around a smaller set of active-learning components. This section is for readers who want to extend the package.

An `OptimizerAL` owns:

- an `ALProposer`, which turns the current archive into a proposed batch;
- one or more surrogates, usually `mlr3mbo::SurrogateLearner` objects;
- acquisition-function prototypes;
- an initialization sampler;
- a result assigner.

`SearchInstance` stores the problem being optimized. It owns the objective, search space, terminator, callbacks, and archive. It supports `"learn"` codomain targets, so a run can collect observations without assigning a single best result.

`OptimizerAL` implements the outer loop. It initializes the archive, asks the proposer for a batch, evaluates the batch, updates callbacks, and repeats until termination. Surrogate fitting and acquisition-function updates are delegated through an `ALContext`. The context represents one proposal round and gives components access to the current archive, candidate pool, pending points, surrogates, and acquisition functions.

The proposers implement different batch-construction policies:

- `ALProposerScore` scores candidates once and takes the top candidates;
- `ALProposerSequentialScore` scores once, then modifies utilities after each within-batch selection;
- `ALProposerSequentialReference` re-scores distance acquisitions after adding selected points as temporary references;
- `ALProposerPseudoLabel` adds pseudo-labels to a temporary archive between within-batch selections;
- `ALProposerPortfolio` combines child proposers round-robin.

Acquisition functions implement the active-learning criterion. The current package includes uncertainty acquisition through standard deviation, GSx for input-space coverage, GSy for output-space diversity, iGS for combined input-output diversity, IDEAL, and QBC through bootstrap standard-error disagreement.

The extension points follow from this decomposition. A new active-learning method can usually be implemented as a new acquisition function, a new proposer, a new score modifier, a new distance object, or a small factory that wires existing components differently.

In-Depth: Space Samplers

Candidate generation and initialization both need to sample points from a search space or from a finite pool. `SpaceSampler` is the shared abstraction for this. It returns a `data.table` with exactly the search-space columns.

The registered sampler ids are:

```
as.data.table(mlr_space_samplers)[, .(key, label, deterministic)]
#> Key: <key>
#>                  key                              label deterministic
#>                <char>                             <char>        <lgcl>
#> 1:             chain            Chained Space Sampler          TRUE
#> 2:       conditional        Conditional Space Sampler          TRUE
#> 3:               gsx                GSx Space Sampler         FALSE
#> 4:            kmeans            K-Means Space Sampler         FALSE
#> 5:          kmedoids          K-Medoids Space Sampler          TRUE
#> 6:               lhs                LHS Space Sampler         FALSE
#> 7: relational_kmeans Relational K-Means Space Sampler         FALSE
#> 8:             sobol              Sobol Space Sampler         FALSE
#> 9:           uniform            Uniform Space Sampler          TRUE
```

The main sampler families are:

- `uniform`, uniform random designs or pool subsamples;
- `lhs`, Latin hypercube designs;
- `sobol`, Sobol sequence designs;
- `gsx`, farthest-point style input-space sampling;
- `kmeans` and `kmedoids`, representative cluster centers or medoids;
- `relational_kmeans`, clustering from relational distances;
- `conditional`, different samplers for discrete and continuous spaces;
- `chain`, sequential composition of samplers.

This gives the direct connection to the practical LHS-candidate workflow. LHS is one sampler choice. Replacing it by Sobol, uniform sampling, GSx, or k-means changes candidate generation but leaves surrogate fitting, acquisition scoring, and batch evaluation unchanged.

In `optimizer_active_learning()`, `acq_optimizer` is translated into a sampler used for acquisition candidates. In `optimizer_pool_al()`, method-specific samplers are used for initialization and optional candidate-pool thinning. Direct `OptimizerAL` construction can use samplers explicitly.

Some samplers need bounded numeric parameters. Some samplers work only on finite pools or only on direct search spaces. The high-level factories choose conservative defaults for the common cases.

In-Depth: Surrogate Models and Uncertainty

Acquisition functions need a uniform prediction interface. The package uses `mlr3mbo::SurrogateLearner` for this. A surrogate binds an `mlr3` learner to an archive, updates the learner from archive data, and exposes predictions to acquisition functions.

Uncertainty-based active learning needs standard errors. There are three ways to obtain them through the user-facing API:

- use a learner with native `"se"` predictions;
- wrap a learner with `LearnerRegrBootstrapSE`;
- wrap a quantile-prediction learner with `LearnerRegrQuantileSE`.

`optimizer_active_learning()` chooses among these through `se_method`. `optimizer_pool_al("qbc")` uses bootstrap disagreement as a query-by-committee score. Direct `OptimizerAL` construction can supply

surrogate objects explicitly.

When surrogate outputs are transformed, scoring must happen on the original target scale. `score_surrogate_on_task()` handles this for `CallbackSurrogatePerformance` and `replay_surrogate_performance()`. This matters because an acquisition function may use transformed-scale predictive moments, while held-out regression measures should be computed on the original output.

The same surrogate has two roles in the report workflow. During active learning, it drives candidate selection. After each batch, its held-out performance becomes the learning curve modelled by `TaskLCE`.

Discussion

The package connects the active-learning run and the learning-curve forecast. The same trace can be used to propose experiments, evaluate surrogate progress, and build a forecasting task. LCE methods are ordinary `mlr3` learners, so they can be resampled, benchmarked, wrapped, and scored with the same tools as other learners. Replay makes traces from expensive workflows available for method development without rerunning the simulator.

Forecast interpretation depends on the held-out task or oracle used for scoring. If the held-out set is unrepresentative, the learning curve describes the wrong performance target. Distributional uncertainty is also method-specific. Parametric curve uncertainty, residual bootstrap uncertainty, conformal uncertainty, and forward-simulation variability answer related but different questions. The benchmark should therefore report both point scores and decision-oriented scores such as target-reaching Brier score.

The practical use is decision support. A target-reaching forecast gives a distribution over additional batches. The forecast can be combined with simulator cost, scientific value of reaching the target, and the feasibility of continuing the experiment.

Conclusion

`celecx` provides infrastructure for learning-curve extrapolation in batch-sequential computer experiments. It covers the full path from active-learning trace generation to `TaskLCE` construction, LCE learner fitting, LCE benchmarking, and batches-to-target forecasting.

For the research project, this gives a concrete framework for evaluating whether an active-learning run should continue and for comparing alternative infill strategies and forecasting methods under the same traces. The small examples in this report demonstrate the package contracts; larger simulator and finite-pool benchmarks use the same code structure.